

Nearside Loopback (RTL Implementation)

High Bandwidth Interface PHY

Exported on 06/03/2025

Table of Contents

1	Implementation Overview.....	4
2	Tx NearSide Loopback	6
2.1	IO Connection.....	6
2.1.1	Two adjacent IOs	6
2.1.2	Full IOs connection with current HW layout.....	8
2.2	Loopback Training	10
2.2.1	Interface	10
2.2.2	Algorithm.....	11
2.2.3	Pattern Comparison.....	12
3	Rx NearSide Loopback	14
3.1	IO Connection.....	14
3.1.1	Daisy-chain connections between IO cells in a column	14
3.1.2	Full IOs connection with current HW layout.....	16
3.2	Loopback Training	18
3.2.1	Interface	18
3.2.2	Algorithm.....	19
3.2.3	Beat Alignment for Pattern comparing.....	20
4	Setup Sequences	23
4.1	Tx Loopback Training.....	23
4.2	Tx Loopback Run	23
4.3	Rx Loopback Training.....	24
5	Loopback Run	26



WIP

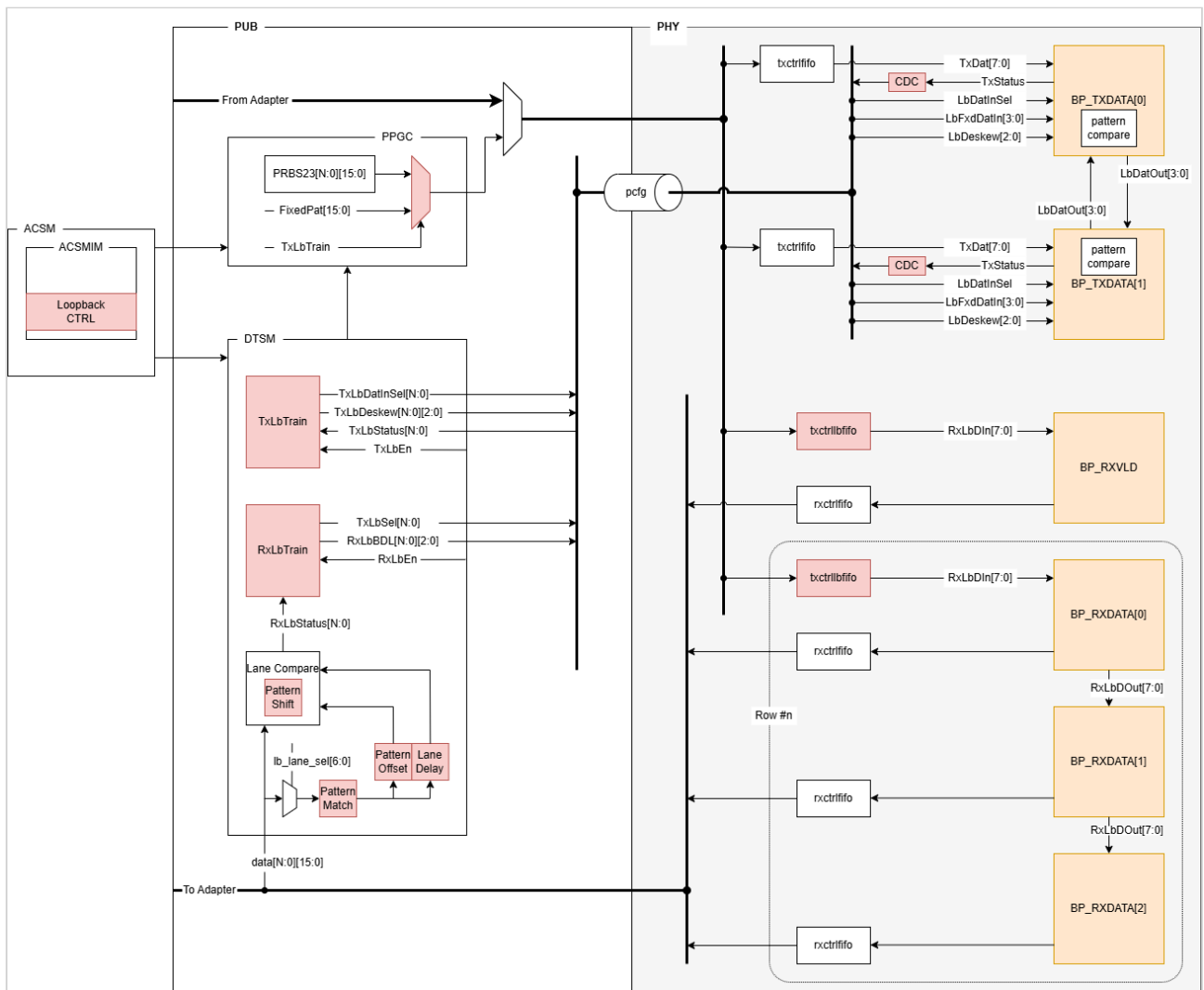
1 Implementation Overview

- Jira discussion [\[P80001562-533260\]](https://jira.internal.synopsys.com/browse/P80001562-533260) [\[UCle Gen 2\] Near-side loopback - Jira \(synopsys.com\)](#)¹
- Arch Spec: [//depot/products/ucie/common/dev_2.0/arch/doc/dwc_uciephy_architecture_specification.pdf](https://depot/products/ucie/common/dev_2.0/arch/doc/dwc_uciephy_architecture_specification.pdf)
 - Section 6.3.2 Loopback BIST Architecture
 - Section 6.3.2 Near-Side Loopback Detail
 - 5.7.2.7 Mainband Transmit Near-Side Loopback Training
 - 5.7.2.8 Mainband Receive Near-Side Loopback Training

RTL Changes:

- DTSM
 - **TxLbTrain:** to support training for Tx Loopback
 - **RxLbTrain:** to support training for Rx Loopback
 - **ValidDelay:** To compensate the delay for Valid signal when choosing a daisy-chain Rx IO
 - Using DTSM command with Loopback enable for training and loopback run
- ACSM
 - Setup ACSM variables for Loopback mode
- PPGC:
 - Add a muxing for fixed pattern and PRBS23 for Tx Training
- Pseudo Config:
 - Add below signals to support Tx Loopback
 - TxStatus: This is status bit and be a result of comparing in Tx loopback, This signal is toggling in 2.5GHz, need to sync to clock domain in PUB
 - LbFxdDatIn[3:0]: Fixed Pattern for Tx Loopback Training, quasi-static config
 - LbDatInSel: Select between Fixed Pattern or PRBS23, quasi-static config
 - LbDeskew[2:0]: adjust BDL for Tx Loopback Training
- PHY UDW:
 - Add a FIFO for each column in IO Cell layout, the RxLbDout[7:0] is connected daisy-chain between RXDATA IOs
 - There are total of 10 columns in layout of Advance package, this means there are 10 FIFOs for them

¹ <https://jira.internal.synopsys.com/browse/P80001562-533260>

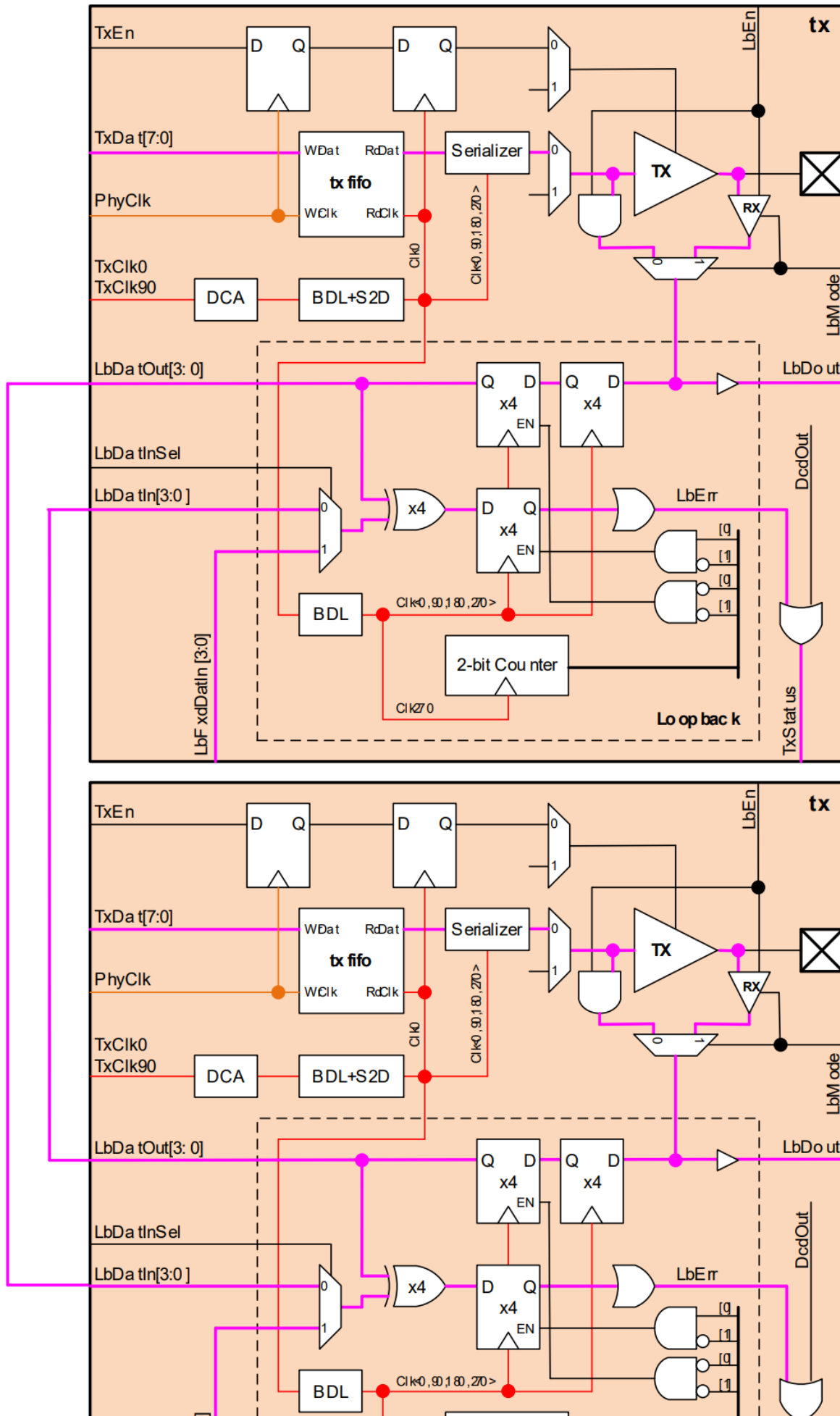


2 Tx NearSide Loopback

2.1 IO Connection

2.1.1 Two adjacent IOs

The detailed connection is described in 6.3.4 Mainband Near-Side Loopback Details of Arch Spec.



2.1.2 Full IOs connection with current HW layout

- Two adjacent IOs with the same color will be connected together.
- Refer to: [dwc_ucie2phy_pub_lb_connection.xlsx](#)², [dwc_ucie2phy_pub_lb_connection_STD.xlsx](#)³ for detail connection
- With current support in PPGC, we have a PRBS23 generator with 8 different seeds to generate 8 PRBS streams, each stream runs on 8 consecutive lanes. This means that we cannot use the different seeds to generate PRBS pattern for Nearside Loopback path with two adjacent IOs. To resolve this and simplify the change, we only support one seed at a time for the whole IO cells and the seed is configurable. [\[Suggestion #1\]](#)

other cell	BP_TX DATA[5]	BP_TX DATA[21]	other cell	BP_TXC KP	other cell	other cell	BP_TX DATA[42]	BP_TX DATA[58]	other cell
BP_TXD ATA[4]	BP_TX DATA[6]	BP_TX DATA[20]	BP_TX DATA[22]	BP_TXC KN	BP_TXT RK	BP_TX DATA[41]	BP_TX DATA[43]	BP_TX DATA[57]	BP_TXD ATA[59]
BP_TXD ATA[3]	BP_TX DATA[7]	BP_TX DATA[19]	BP_TX DATA[23]	BP_TXC KRD	BP_TXV LD	BP_TX DATA[40]	BP_TX DATA[44]	BP_TX DATA[56]	BP_TXD ATA[60]
BP_TXD ATA[2]	BP_TX DATA[8]	BP_TX DATA[18]	BP_TX DATA[24]	other cell	BP_TXV LDRD	BP_TX DATA[39]	BP_TX DATA[45]	BP_TX DATA[55]	BP_TXD ATA[61]
BP_TXD ATA[1]	BP_TX DATA[9]	BP_TX DATA[17]	BP_TX DATA[25]	BP_TXD ATARD[1]	BP_TXD ATARD[2]	BP_TX DATA[38]	BP_TX DATA[46]	BP_TX DATA[54]	BP_TXD ATA[62]

² [https://synopsys.sharepoint.com/:x:/s/h200-ucie-a-ss4lpp-12/](https://synopsys.sharepoint.com/:x:/s/h200-ucie-a-ss4lpp-12/EVhg9Cf4fJNliuUWnRyvAiQBnXndM65XvKt39WxKlIMzJw?e=ZaV9II&clickparams=eyJBCbHOYW1lIjoIbGVhbmMtRGVza3RvcCIsIkFwcFZlcnNpb24iOiI0OS8yNDEwMjAwMTMxOCIsIkhhc0ZlZGVyYXRIZFVzZXIiOiI0OmZhbHNlIiwiaWF0IjpmQ%3D%3D)

[EVhg9Cf4fJNliuUWnRyvAiQBnXndM65XvKt39WxKlIMzJw?](https://synopsys.sharepoint.com/:x:/s/h200-ucie-a-ss4lpp-12/EVhg9Cf4fJNliuUWnRyvAiQBnXndM65XvKt39WxKlIMzJw?e=ZaV9II&clickparams=eyJBCbHOYW1lIjoIbGVhbmMtRGVza3RvcCIsIkFwcFZlcnNpb24iOiI0OS8yNDEwMjAwMTMxOCIsIkhhc0ZlZGVyYXRIZFVzZXIiOiI0OmZhbHNlIiwiaWF0IjpmQ%3D%3D)

[e=ZaV9II&clickparams=eyJBCbHOYW1lIjoIbGVhbmMtRGVza3RvcCIsIkFwcFZlcnNpb24iOiI0OS8yNDEwMjAwMTMxOCIsIkhhc0ZlZGVyYXRIZFVzZXIiOiI0OmZhbHNlIiwiaWF0IjpmQ%3D%3D](https://synopsys.sharepoint.com/:x:/s/h200-ucie-a-ss4lpp-12/EVhg9Cf4fJNliuUWnRyvAiQBnXndM65XvKt39WxKlIMzJw?e=ZaV9II&clickparams=eyJBCbHOYW1lIjoIbGVhbmMtRGVza3RvcCIsIkFwcFZlcnNpb24iOiI0OS8yNDEwMjAwMTMxOCIsIkhhc0ZlZGVyYXRIZFVzZXIiOiI0OmZhbHNlIiwiaWF0IjpmQ%3D%3D)

³ [https://synopsys.sharepoint.com/:x:/s/h200-ucie-a-ss4lpp-12/](https://synopsys.sharepoint.com/:x:/s/h200-ucie-a-ss4lpp-12/EUaBPYhYCBBKruBgV717sLsBHWEisPLdHjeKuZKdIZs2BQ?e=sAYBU)

[EUaBPYhYCBBKruBgV717sLsBHWEisPLdHjeKuZKdIZs2BQ?e=sAYBU](https://synopsys.sharepoint.com/:x:/s/h200-ucie-a-ss4lpp-12/EUaBPYhYCBBKruBgV717sLsBHWEisPLdHjeKuZKdIZs2BQ?e=sAYBU)

BP_TXD ATA[0]	BP_TX DATA[1 0]	BP_TX DATA[1 6]	BP_TX DATA[2 6]	BP_TXD ATA[31]	BP_TXD ATA[32]	BP_TX DATA[3 7]	BP_TX DATA[4 7]	BP_TX DATA[5 3]	BP_TXD ATA[63]
BP_TXD ATA[12]	BP_TX DATA[1 1]	BP_TX DATA[1 5]	BP_TX DATA[2 7]	BP_TXD ATA[30]	BP_TXD ATA[33]	BP_TX DATA[3 6]	BP_TX DATA[4 8]	BP_TX DATA[5 2]	BP_TXD ATA[51]
BP_TXD ATARD[0]	BP_TX DATA[1 3]	BP_TX DATA[1 4]	BP_TX DATA[2 8]	BP_TXD ATA[29]	BP_TXD ATA[34]	BP_TX DATA[3 5]	BP_TX DATA[4 9]	BP_TX DATA[5 0]	BP_TXD ATARD[3]

IO Name	IO Index
BP_TXDATA[63:0]	63:0
BP_TXDATARD[0]	64
BP_TXDATARD[1]	65
BP_TXDATARD[2]	66
BP_TXDATARD[3]	67
BP_TXVLD	68
BP_TXVLDRD	69
BP_TXCKP	70
BP_TXCKN	71
BP_TXTRK	72
BP_TXCKRD	73

2.2 Loopback Training

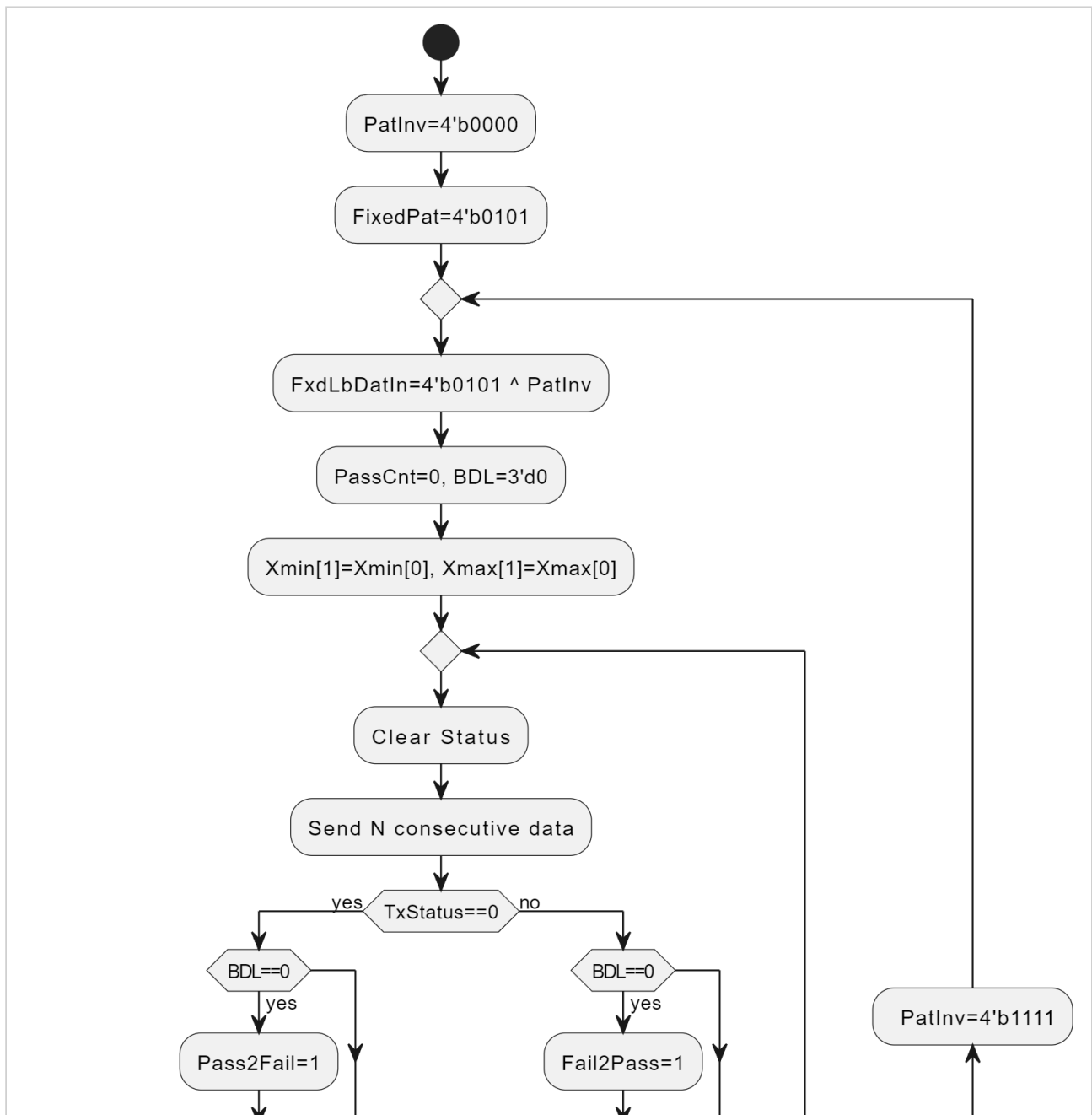
2.2.1 Interface

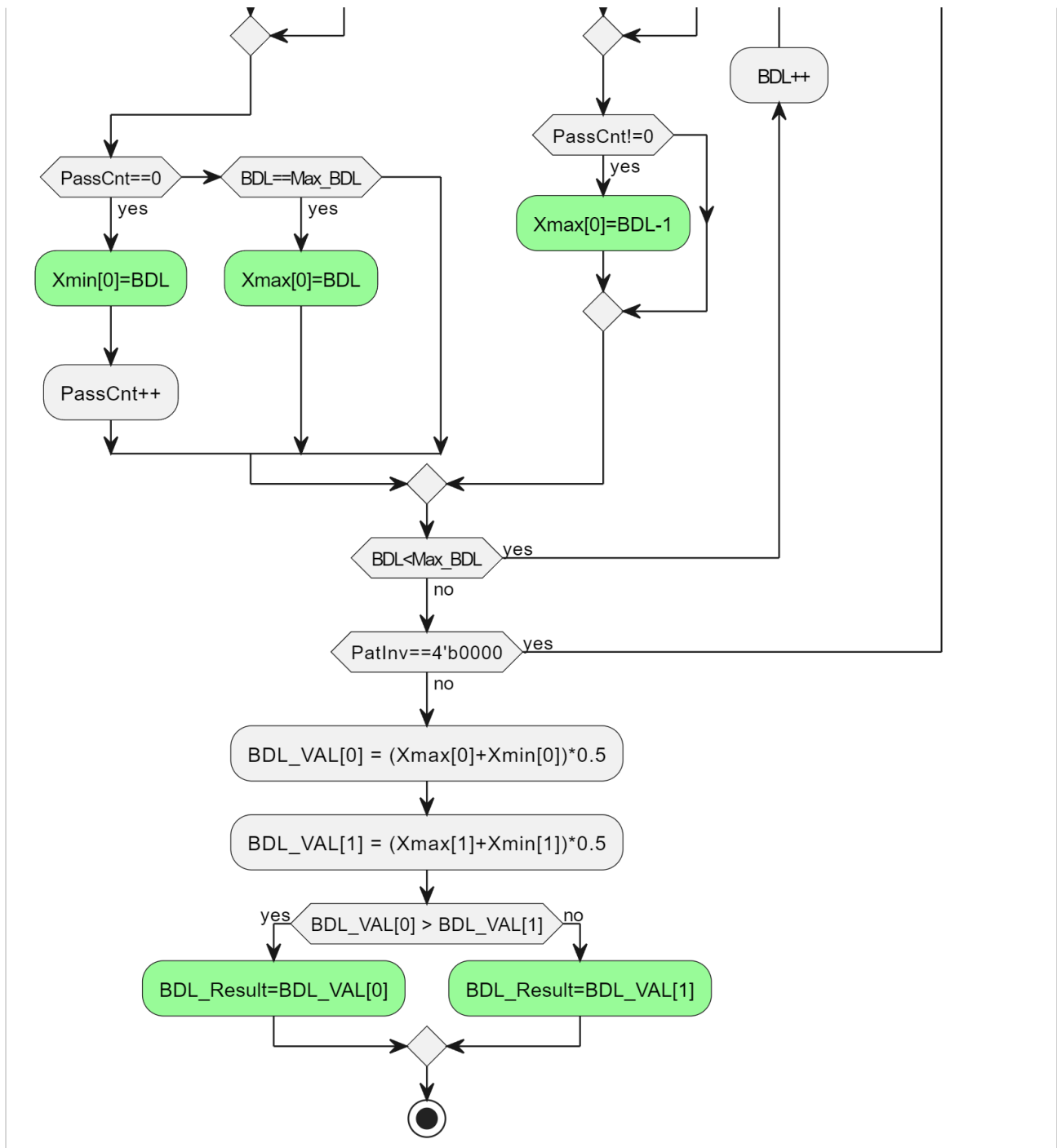
IO	Direction	Description
Ctlclk	IN	2.5 GHz Clock
Rresetn	IN	Active low reset
i_lbtrain_trig	IN	Enable Tx Loopback training
i_lbtrain_lstr	IN	Command to start training
i_lbtrain_lend	IN	Command to end training
i_txstatus[TX_LANE_NUM-1:0]	IN	Pattern compare status from PHY DW
i_dwtxin[TX_LANE_NUM-1:0]	IN	Lane enable
o_lbtrain_done	OUT	Indicate training is done
o_lb_req	OUT	Request to update BDL value to IO cells
o_lb_inf[\$bits(txlb_pcfg_t)-1:0]	OUT	IO cell settings: <pre>typedef struct packed { logic patset; //1:IO cells compare with fixed pattern; 0:PRBS23 pattern logic [3:0] fixpat; //1:Fixed Pattern; 0:PRBS23 Pattern logic [pLB_BDL_B-1:0] bdlval; //BDL vaule for loopback } txlb_pcfg_t;</pre>

IO	Direction	Description
i_lb_ack	IN	Settings have been configured to IO cells

2.2.2 Algorithm

The algorithm for Tx Near-Side loopback Training is described in section 5.7.2.7 *Mainband Transmit Near-Side Loopback Training* of Architecture Specification





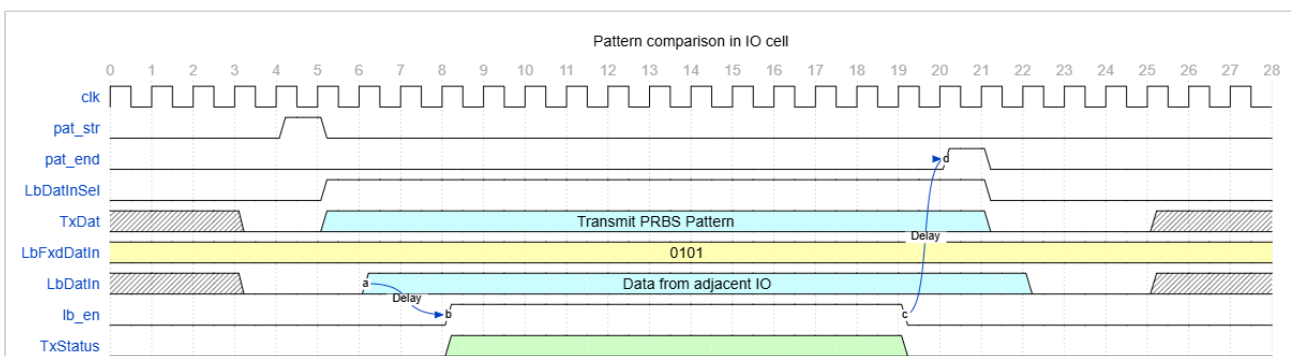
The signal TxStatus is synced from PHY DW to PUB DW by a synchronizer located in PUB

BDL settings include Pattern_Select, Fixed_Pattern, BDL_Value are sent from training engine in PUB/DTSM/* DW to PHY DW via Pseudo Configuration

2.2.3 Pattern Comparison

- Loopback Training phase

- Purpose: The training phase is designed to ensure proper alignment and calibration of each IO cell.
- Process:
 - A fixed pattern is generated by the Training module. The self-comparator within each IO cell compares this fixed pattern with its corresponding tied pattern
 - The comparison results are recorded in the TxStatus. This allows for the training module do further analysis and update BDL value for each IO cell.
- Loopback Run phase
 - Purpose: The run phase focuses on verifying data integrity between adjacent IO cells.
 - Process:
 - A pattern is generated by the PPGC.
 - The self-comparator compares the generated pattern between two adjacent IO cells.
 - The results of this comparison are sent to the TxStatus of both IO cells involved in the process.
- Comparison timing in each IO cell
 - Data pattern is continuously compared in each IO cell and recorded in TxStatus with a LbEn signal. There are no ways capture this signal aligned with the timing PPGC transmit the pattern. To avoid this issue, there are two ways to get a properly TxStatus signal:
 - The training module will assert LbEn signal after generating pattern in PPGC. This leads to some first and last pattern will be not compared in IO cell. Currently, our design is comparing 1bit in 4 received bits. So this way does not significantly change the meaning of comparison.
 - Another way to capture TxStatus is that base on a CSR setting delay time from PPGC to IO cells. This mean that the TxStatus will be captured after some delay from PPGC start.



3 Rx NearSide Loopback

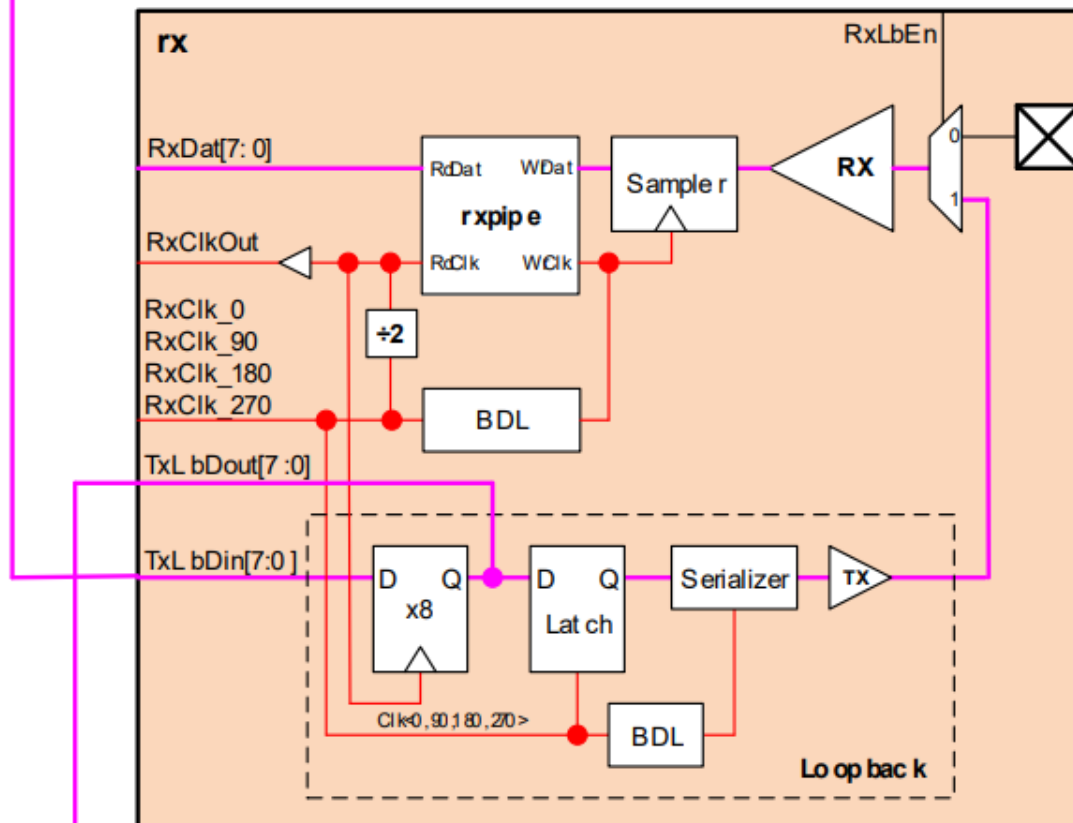
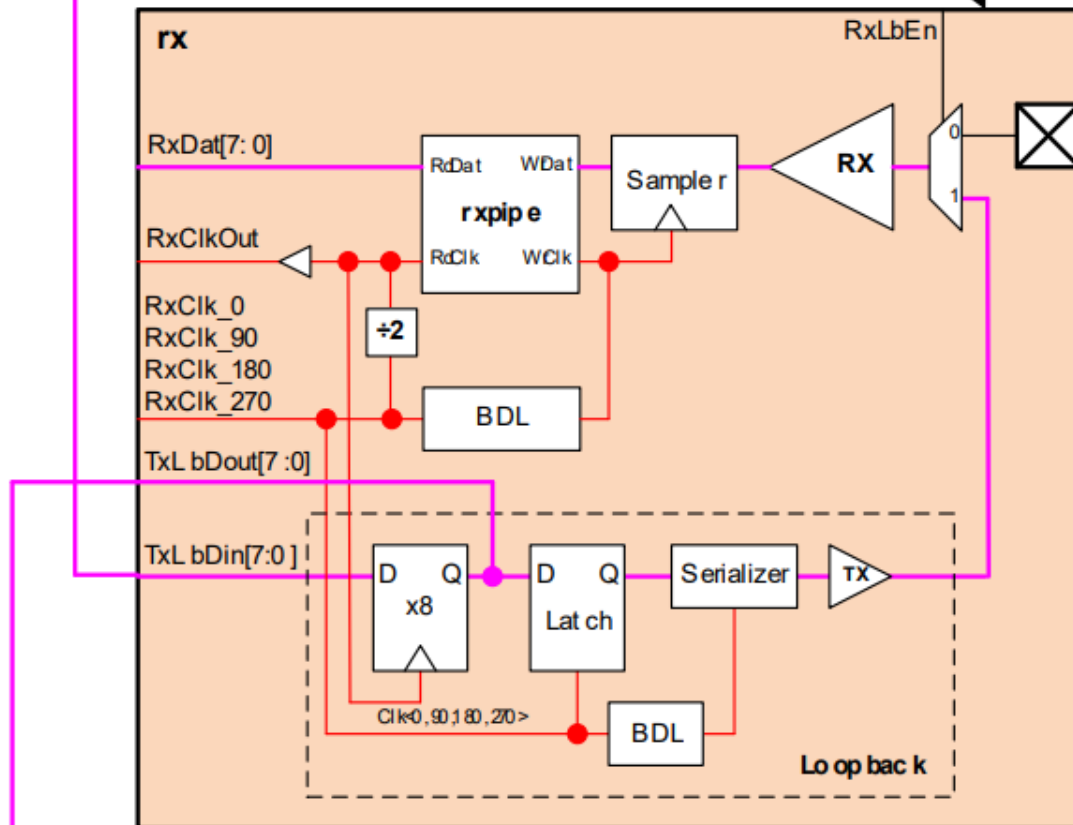
3.1 IO Connection

3.1.1 Daisy-chain connections between IO cells in a column

- The IO cells belong to a column in HW layout will be connected as a daisy-chain
- The txctrlbfifo is the first chain, it receives data from PPGC and send to the remaining chains
- The last chain is different between column, it depends on the IO layout

From txclb fifo

rx clb test to
txclb fifo



To next
adjacent rx

1 Two adjacent IO connection

3.1.2 Full IOs connection with current HW layout

- There are ten columns of IO cells
- The same color is a chain
- Refer to: [dwc_ucie2phy_pub_lb_connection.xlsx](#)⁴ for detail connection
- This constrain in IO connections make the Rx NearSide Loopback has the same limitation as Tx NearSide Loopback:
 - To resolve this and simplify the change, we only support one seed at a time for the whole IO cells and the seed is configurable. **The same as [Suggestion #1]**
 - BP_RXVLD and BP_RXVLD RD work as normal IO cells when running loopback, so we can not use them for qualifying data lanes. To solve this we should perform following conditions: **[Suggestion #2]**
 - Loopback Training must run on continuous clock mode so that there is no reset and FIFO calibration during running.
 - Data and Valid lanes should be held low if not performing any training or loopback run.
 - PPGC will support per lane alignment based on the configuration of seed. Refer to [BeatAlignmentforPatterncomparing](#) (see page 20) for detail implementation

txctrlbfi fo[0]	txctrlb fifo[1]	txctrlb fifo[2]	txctrlb fifo[3]	txctrlbfi fo[4]	txctrlbfi fo[5]	txctrlb fifo[6]	txctrlb fifo[7]	txctrlb fifo[8]	txctrlbfi fo[9]
BP_RXD ATARD[3]	BP_RX DATA[50]	BP_RX DATA[49]	BP_RX DATA[35]	BP_RXD ATA[34]	BP_RXD ATA[29]	BP_RX DATA[28]	BP_RX DATA[14]	BP_RX DATA[13]	BP_RXD ATARD[0]
BP_RXD ATA[51]	BP_RX DATA[52]	BP_RX DATA[48]	BP_RX DATA[36]	BP_RXD ATA[33]	BP_RXD ATA[30]	BP_RX DATA[27]	BP_RX DATA[15]	BP_RX DATA[11]	BP_RXD ATA[12]

⁴<https://synopsys.sharepoint.com/:x:/s/h200-ucie-a-ss4lpp-12/EVhg9Cf4fJNliuUWnRyvAiQBnXndM65XvKt39WxKlImzJw?e=ZaV9II&clickparams=eyJBCtHBOYW1IjoiVGhVhbXMtRGVza3RvcCIsIkFwcFZlcnNpb24iOiI0OS8yNDEwMjAwMTMxOCIsIkhhc0ZlZGVyYXRlZFVzZXIiOiI0MzhbHNIHQ%3D%3D>

BP_RXD ATA[63]	BP_RX DATA[53]	BP_RX DATA[47]	BP_RX DATA[37]	BP_RXD ATA[32]	BP_RXD ATA[31]	BP_RX DATA[26]	BP_RX DATA[16]	BP_RX DATA[10]	BP_RXD ATA[0]
BP_RXD ATA[62]	BP_RX DATA[54]	BP_RX DATA[46]	BP_RX DATA[38]	BP_RXD ATARD[2]	BP_RXD ATARD[1]	BP_RX DATA[25]	BP_RX DATA[17]	BP_RX DATA[9]	BP_RXD ATA[1]
BP_RXD ATA[61]	BP_RX DATA[55]	BP_RX DATA[45]	BP_RX DATA[39]	BP_RXV LDRD	other cell	BP_RX DATA[24]	BP_RX DATA[18]	BP_RX DATA[8]	BP_RXD ATA[2]
BP_RXD ATA[60]	BP_RX DATA[56]	BP_RX DATA[44]	BP_RX DATA[40]	BP_RXV LD	BP_RXC KRD	BP_RX DATA[23]	BP_RX DATA[19]	BP_RX DATA[7]	BP_RXD ATA[3]
BP_RXD ATA[59]	BP_RX DATA[57]	BP_RX DATA[43]	BP_RX DATA[41]	BP_RXT RK	BP_RXC KN	BP_RX DATA[22]	BP_RX DATA[20]	BP_RX DATA[6]	BP_RXD ATA[4]
other cell	BP_RX DATA[58]	BP_RX DATA[42]	other cell	other cell	BP_RXC KP	other cell	BP_RX DATA[21]	BP_RX DATA[5]	other cell

IO Name	IO Index
BP_RXDATA[63:0]	63:0
BP_RXDATARD[0]	64
BP_RXDATARD[1]	65
BP_RXDATARD[2]	66

IO Name	IO Index
BP_RXDATARD[3]	67
BP_RXVLD	68
BP_RXVLD RD	69

3.2 Loopback Training

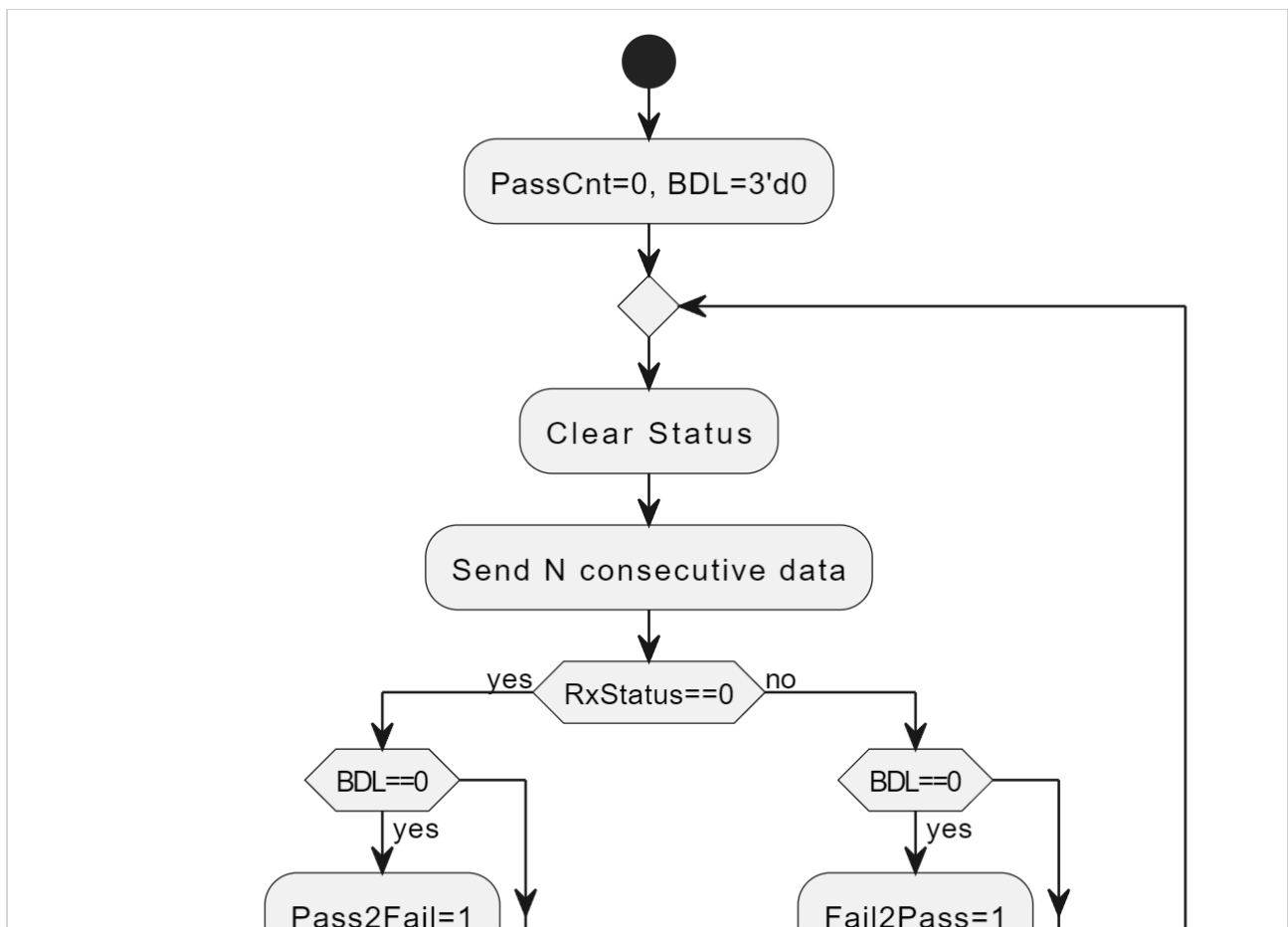
3.2.1 Interface

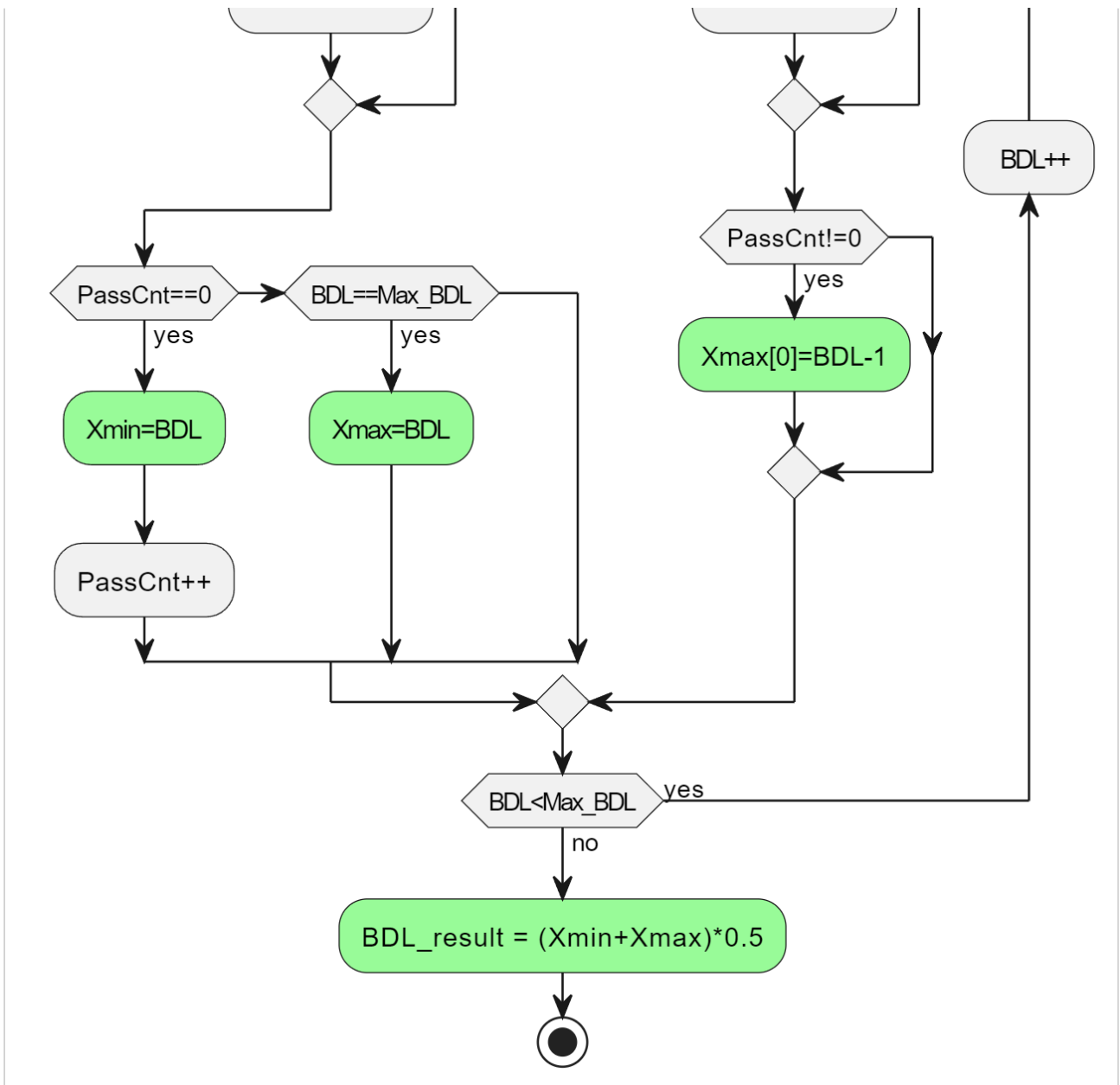
IO	Direction	Description
Ctlclk	IN	2.5 GHz Clock
Rresetn	IN	Active low reset
i_lbtrain_trig	IN	Enable Tx Loopback training
i_lbtrain_lstr	IN	Command to start training
i_lbtrain_lend	IN	Command to end training
i_rxstatus[TX_LANE_NUM-1:0]	IN	Pattern compare status from DTSM COMP module
o_lbtrain_done	OUT	Indicate training is done
o_lb_req	OUT	Request to update BDL value to IO cells

IO	Direction	Description
o_lb_inf[\$bits(rxlb_pcfg_t)-1:0]	OUT	IO cell settings: <pre>typedef struct packed { logic [pLB_BDL_B-1:0] bdlval; //BDL value for loopback } rxlb_pcfg_t;</pre>
i_lb_ack	IN	Settings have been configured to IO cells

3.2.2 Algorithm

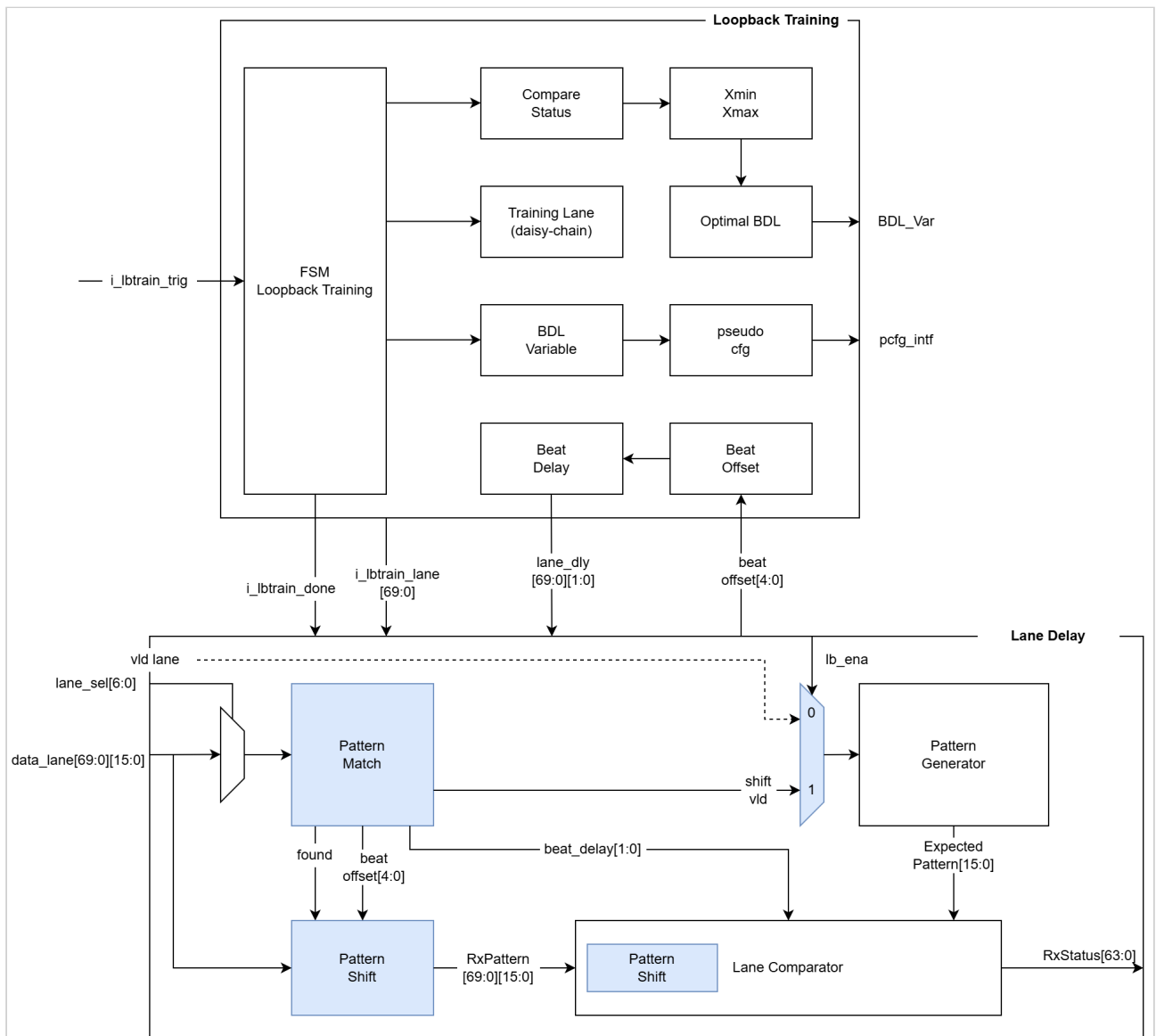
The algorithm for Rx Near-Side loopback Training is described in section 5.7.2.8 Mainband Receive Near-Side Loopback Training of Architecture Specification



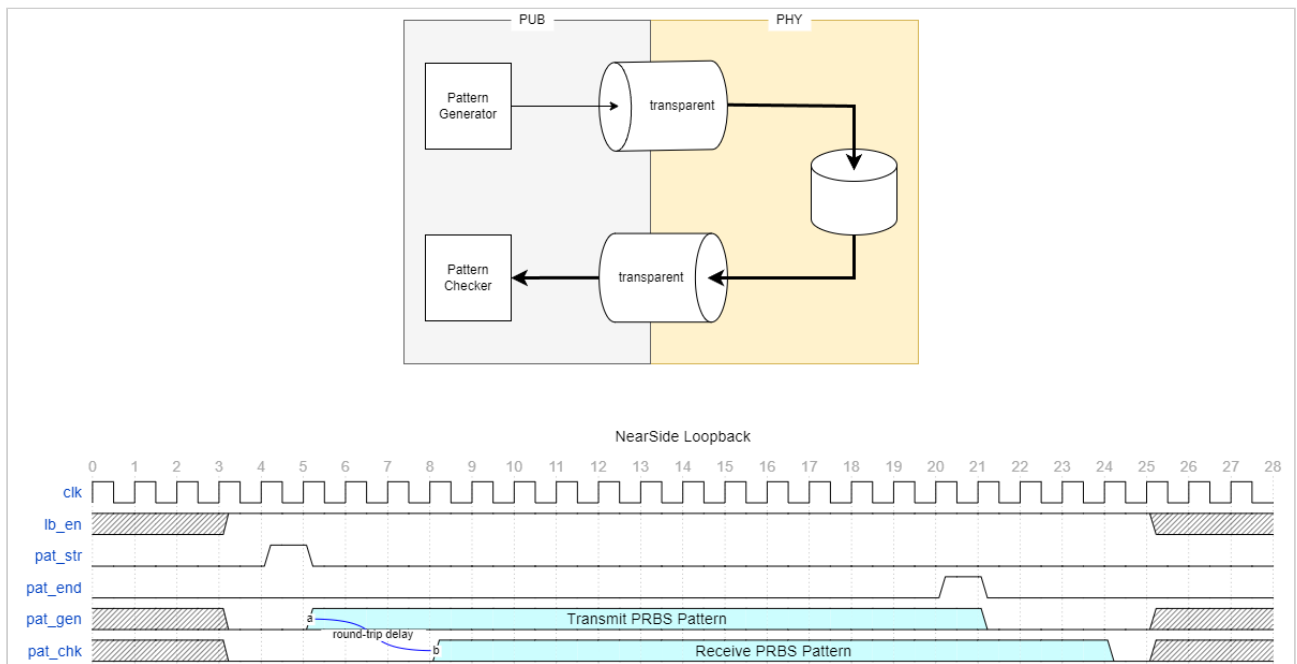


3.2.3 Beat Alignment for Pattern comparing

In Rx nearside loopback, the received data for each lane has different delays due to the daisy-chain connection between the RxIO Cells in the PHY block. So we cannot use the valid lane for qualifying data between lanes because the skews of them. In order to support comparing data, we need to detect the valid pattern for each lane and align them to a common point to fit in 16bit data. This timing will be used to generate the expected pattern and send to the comparator.



To ensure the pattern detector works properly, the clock should be in free running mode and data lanes in main-band are driven to low when not performing training or loopback function. The following timing is requirement for Rx Nearside loopback mode:



4 Setup Sequences

4.1 Tx Loopback Training

1. Setup Tx Loopback mode
 - a. At the beginning, Setup ACSM so that LTSM will go from RESET, SBINIT, SPEEDIDLE, TXSELFCAL, to RXCLKCAL
 - b. Setup Sideband loopback
 - i. `DwLoopBackTxEn = 1'b1`
 - ii. `DwLoopBackRxEn = 1'b1`
 - iii. `DwNearSideLoopBack = 1'b1`
 - iv. `DwLoopbackTrain = 1'b0`
 - c. Start ACSM
 - d. Wait AcsmDone
 - e. Write DwModeCtrl0 register with following values:
 - i. `DwNearSideLoopBack = 1'b1`
 - ii. `DwLoopBackRxEn = 1'b0`
 - iii. `DwLoopBackTxEn = 1'b1` for enabling Tx Loopback of IO cells
 - iv. `DwLoopBackMode = 0` for pre-driver loopback BIST, 1 for post-driver loopback BIST
 - v. `DwLoopbackTrain = 1` for loopback training
 - vi. `DwLoopBack = 1`
 - f. Setup ACSM so that LTSM will only run at LINKSPEED
 - g. Write AcsmLtismMsk0Var19 to 0C210003
 - h. Write AcsmLtismMsk1Var19 to 00020000
 - i. Write AcsmCtrl.AcsmStartAddr = 0
 - j. Write AcsmInstrReg16 to 23a11003
 - k. Start ACSM
 - l. Wait DwCurrEdgeDone for Training Done
2. LTSM states in Loopback mode
 - a. RESET
 - b. SBINIT
 - c. SPEEDIDLE
 - d. TXSELFCAL
 - e. RXCLKCAL
 - f. LINKSPEED (Train & Run loopback)

4.2 Tx Loopback Run

1. At the LINKSPEED state

2. Write AcsmCtrl.AcsStartAddr = 0
3. Start ACSM
4. Wait AcsmDone
5. Check status on DwDtsmLnErr0/DwDtsmLnErr1/DwDtsmLnErr2

4.3 Rx Loopback Training

1. Setup Tx Loopback mode
 - a. At the beginning, Setup ACSM so that LTSM will go from RESET, SBINIT, SPEEDIDLE, TXSELF CAL, to RXCLKCAL
 - b. Setup Sideband loopback
 - i. `DwLoopBackTxEn = 1'b1`
 - ii. `DwLoopBackRxEn = 1'b1`
 - iii. `DwNearSideLoopBack = 1'b1`
 - iv. `DwLoopbackTrain = 1'b0`
 - c. Start ACSM
 - d. Wait AcsmDone
 - e. Write DwModeCtrl0 register with following values:
 - i. `DwNearSideLoopBack = 1'b1`
 - ii. `DwLoopBackTxEn = 1'b0`
 - iii. `DwLoopBackRxEn = 1'b1`
 - iv. `DwLoopbackTrain = 1` for loopback training
 - v. `DwLoopBack = 1`
 - f. Setup ACSM so that LTSM will only run at LINKSPEED
 - g. Write AcsmLtsmMsk0Var19 to 32'h0C210003
 - h. Write AcsmLtsmMsk1Var19 to 32'h00020000
 - i. Write AcsmCtrl.AcsStartAddr = 0
 - j. Write AcsmInstrReg16 to 23a11003
 - k. Start ACSM
 - l. Wait DwCurrEdgeDone for Training Done
2. LTSM states in Loopback mode
 - a. RESET
 - b. SBINIT
 - c. SPEEDIDLE
 - d. TXSELF CAL
 - e. RXCLKCAL
 - f. LINKSPEED (Train & Run loopback)

```
i. assign lane_arr[] = {
    7'd67, 7'd51, 7'd63, 7'd62, 7'd61, 7'd60,
    7'd59,          //Chain#0
```



```

7'd50, 7'd52, 7'd53, 7'd54, 7'd55, 7'd56, 7'
d57, 7'd58, //Chain#1
7'd49, 7'd48, 7'd47, 7'd46, 7'd45, 7'd44, 7'
d43, 7'd42, //Chain#2
7'd35, 7'd36, 7'd37, 7'd38, 7'd39, 7'd40,
7'd41, //Chain#3
7'd34, 7'd33, 7'd32, 7'd66, 7'd69, 7'd68,
7'd29, 7'd30, 7'd31, 7'd65,
7'd28, 7'd27, 7'd26, 7'd25, 7'd24, 7'd23,
7'd22,
7'd14, 7'd15, 7'd16, 7'd17, 7'd18, 7'd19, 7'
d20, 7'd21,
7'd13, 7'd11, 7'd10, 7'd09, 7'd08, 7'd07, 7'
d06, 7'd05,
7'd64, 7'd12, 7'd00, 7'd01, 7'd02, 7'd03,
7'd04
};

//Select lane per chain for training
always_ff @(posedge clk) train_lane[7:0] <= train_perlane_done ?
(lane_arr << lane_count*7) : train_lane[7:0];

```

5 Loopback Run

- At the LINKSPEED state
- Write AcsmCtrl.AcsmStartAddr = 0
- Start ACSM
- Wait AcsmDone
- Check status on DwDtsmLnErr0/DwDtsmLnErr1/DwDtsmLnErr2

Action items:

- Bao to update DOC to confirm with Peter
- Duy to check VREF, Rx Deskew training flows with CKT team